

Algorithmen und Datenstrukturen



Willkommen

Moodle-Kurs

- Kursname „Algorithmen und Datenstrukturen“
- Selbsteinschreibung mit dem Kennwort
4LG0D4T

Oliver Hofmann

- Professor an der TH Nürnberg seit 2015, zuvor an der TH Würzburg/Schweinfurt seit 2012
- Dipl. Informatiker
- Modulverantwortlicher "Software Engineering"
- Studiengangsleiter "Media Engineering"

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppenarbeit

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppe 1:

Welche der beiden Anleitungen ist ein Algorithmus?

1. Setze $i \leftarrow 1$
2. Addiere 1 zu i
3. Addiere 1 zu i
4. Addiere 1 zu i

...

1. Setze $i \leftarrow 1$
2. Addiere 1 zu i
3. Wiederhole Schritt 2 solange $i < 10$

Welche Eigenschaft fordern wir daher von einem Algorithmus?

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppe 2:

Welche der beiden Anleitungen ist ein Algorithmus?

1. Gib n ein
2. Wenn $n < 1$ gehe zu Schritt 4
3. Subtrahiere 1 von n
4. Gehe zu Schritt 2
5. Gib n aus

1. Gib n ein
2. Wenn $n < 1$ gehe zu Schritt 5
3. Subtrahiere 1 von n
4. Gehe zu Schritt 2
5. Gib n aus

Welche Eigenschaft fordern wir daher von einem Algorithmus?

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppe 3:

Welche der beiden Anleitungen ist ein Algorithmus?

1. Nimm Brot.
2. Lege eine angemessene Menge Käse darauf.
3. Erwärme das Sandwich ein bisschen.
4. Serviere es schön.

1. Nimm 1 Scheibe Brot
2. Lege zwei Scheiben Käse darauf
3. Erwärme das Sandwich auf 80 Grad
4. Serviere es warm auf einem Teller

Welche Eigenschaft fordern wir daher von einem Algorithmus?

Was ist ein Algorithmus?

Merkmale eines Algorithmus

Gruppe 4:

Welche der beiden Anleitungen ist ein Algorithmus?

1. Gib aus "3 + 5".
2. Gib aus " = 8"

1. Gib a und b ein
2. Addiere a und b und gib das Ergebnis aus

Welche Eigenschaft fordern wir daher von einem Algorithmus?

Was ist ein Algorithmus?

Ein Algorithmus ist

- eine **endliche Folge** von Regeln,
- nach denen sich nach **endlich vielen**,
- **eindeutig festgelegten** Schritten
- die Lösung einer **Klasse von Problemen** ergibt.

Finitheit

Wir können den Algorithmus mit endlichen Ressourcen beschreiben

Terminiertheit

Der Algorithmus endet nach endlicher Zeit

Determiniertheit

Es gibt keinen Interpretationsspielraum

Allgemeingültigkeit

Lösung wird für unterschiedliche Eingaben errechnet

Was ist eine Datenstruktur?

Eine Datenstruktur ist

- Die Art und Weise, die die von einem Algorithmus bearbeiteten Daten organisiert und vorgehalten werden.
- Algorithmen und Datenstrukturen bedingen sich gegenseitig, d.h. für die Durchführung eines Algorithmus ist es i.d.R. notwendig, dass die Daten in einer bestimmten Datenstruktur vorliegen.
- Manchmal ist das Erzeugen der Datenstruktur aufwändiger als die anschließende Durchführung des Algorithmus.

Beispiel

30 Zahlen $\{a_1 \dots a_{30}\}$ zwischen 1 und 250 sollen der Größe nach sortiert werden beginnend bei der größten Zahl.

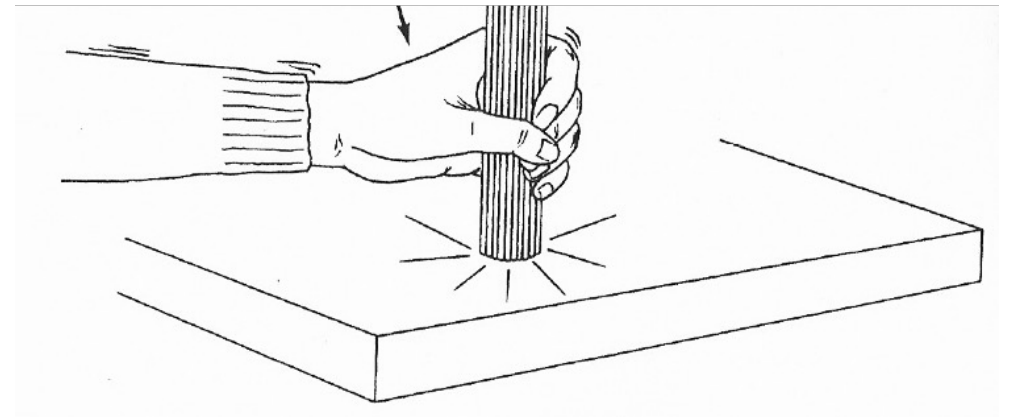
Vorbereiten der Datenstruktur

Für jede Zahl a_i wird eine Spagetti-Nudel auf die Länge $\frac{a_i}{10}$ cm gekürzt.

Durchführen des Algorithmus

Das Bündel Spagetti wird auf den Tisch geschlagen und sukzessive immer die längste Nudel herausgezogen und ihre Länge notiert.

Die Vorbereitung ist hier sicher aufwändiger als die Durchführung!



Ziele der Vorlesung

- Wir lernen für eine Reihe von Problemklassen unterschiedliche Algorithmen mit ihren Datenstrukturen kennen und vergleichen sie.
- Einige der Algorithmen werden wir in den Praktika implementieren, um die jeweiligen Herausforderungen besser zu verstehen.
- Am Ende sollten wir in der Lage sein, für neue und unbekannte Problemklassen brauchbare Algorithmen entwerfen zu können.

Disclaimer: Viele der von uns betrachteten Algorithmen sind bereits in Klassenbibliotheken der unterschiedlichen Programmiersprache enthalten. Das ändert aber nichts daran, dass man sie verstanden haben sollte, um die Lösungsansätze auf neue Problemstellungen anwenden zu können!

Wie stellen wir einen Algorithmus dar?

Wie stellen wir einen Algorithmus dar?

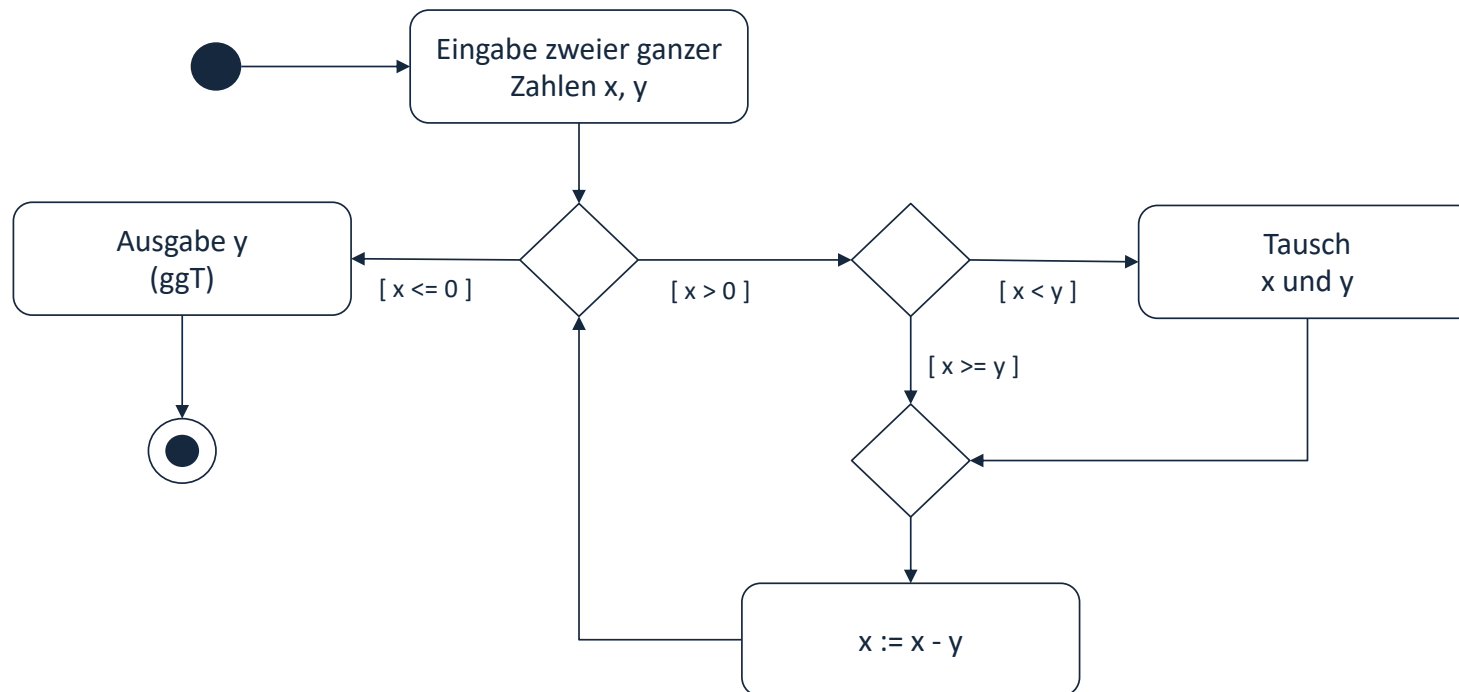
Informelle Darstellung eines Algorithmus

1. Lies zwei ganze Zahlen in die Variablen `,x'` und `,y'` ein!
2. Solange der Wert von `,x'` größer als 0 ist, wiederhole die Schritte 2.1 und 2.2
 - 2.1 Wenn der Wert von `,x'` kleiner als `,y'`, dann vertausche die Inhalte von `,x'` und `,y'`!
 - 2.2 Weise `,x'` die Differenz von `,x'` minus `,y'` zu!
3. Gib den Wert von `,y'` als ggT aus!

Es handelt sich hierbei um den Euklid-Algorithmus zur Berechnung des größten gemeinsamen Teilers (ggT) zweier ganzer Zahlen.

Wie stellen wir einen Algorithmus dar?

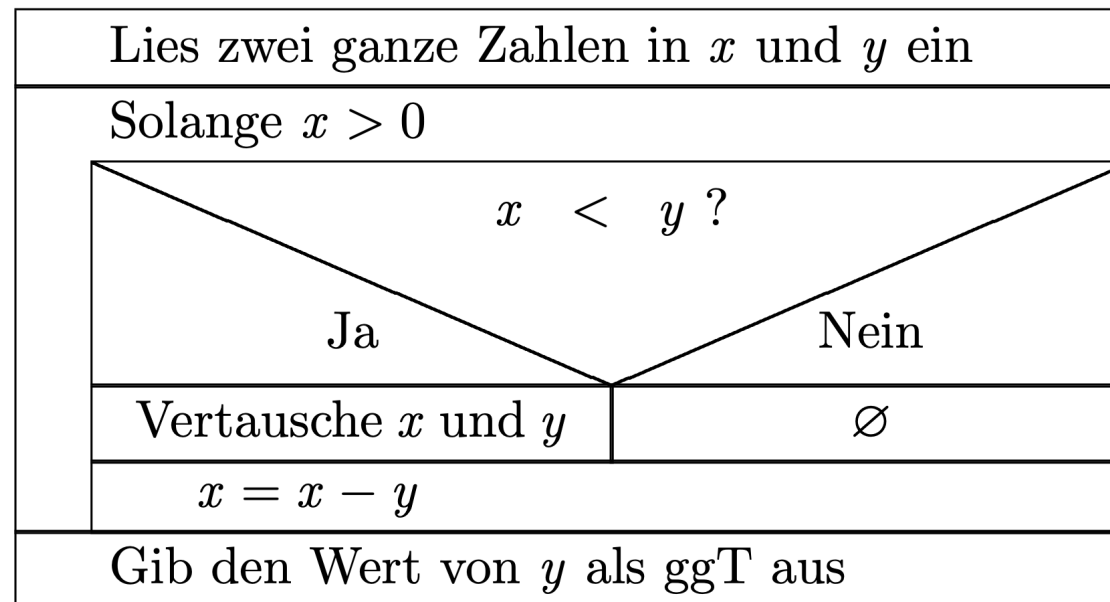
Darstellung als UML Aktivitätsdiagramm



Wie stellen wir einen Algorithmus dar?

Darstellung als Struktogramm von Nassi und Shneiderman (DIN 66261)

Euklid-Algorithmus



Wie stellen wir einen Algorithmus dar?

Darstellung als Pseudocode

Euklid-Algorithmus

Ensure: $x, y \in \mathbb{N}$

```
1: while  $x > 0$  do  
2:   if  $x < y$  then  
3:      $temp \leftarrow x$   
4:      $x \leftarrow y$   
5:      $y \leftarrow temp$   
6:   end if  
7:    $x \leftarrow x - y$   
8: end while  
9: return  $y$ 
```

▷ y enthält den gesuchten ggT

- Ein Pseudocode ist eine semi-formale Sprache, mit der man die Ablaufstrukturen eines Programms grob durch Texte und fest vorgegebene Schlüsselwörter beschreiben kann.
- Die Schlüsselwörter lehnen sich in der Regel an die einer höheren Programmiersprache an.

Implementieren eines Algorithmus

Definition

Das Formulieren eines Algorithmus in einer maschinenlesbaren Form, so dass der Algorithmus von einem Rechner ausgeführt werden kann, nennt man Implementieren des Algorithmus.

Der entstehende Programmcode ist ebenfalls eine Darstellung des Algorithmus, jedoch enthält er i.d.R. weitergehende Implementierungsdetails, die nicht Kernbestandteil des Algorithmus sind.

Implementieren eines Algorithmus

Beispiel-Implementierung in Python

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$	
Ja	Nein
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	



```
def euklid_ggt(x, y):
    while x > 0:
        if x < y:
            (x, y) = (y, x)
        x -= y
    return y
```

Implementierungsdetail:
Variablentausch mit Hilfe von Tupeln

Eigenschaften von Algorithmen

Eigenschaften von Algorithmen

In dieser Veranstaltung interessieren uns vorrangig zwei Eigenschaften:

Korrektheit

Ein Algorithmus ist korrekt, wenn er die Aufgabe richtig löst.

Effizienz

Ein Algorithmus ist effizient, wenn er vergleichsweise wenige Schritte/Operationen und wenig Speicher benötigt.



Diese Eigenschaften sind nicht abhängig von der Implementierung!
Ein Algorithmus wird nicht korrekter oder effizienter durch die Nutzung einer anderen Programmiersprache!

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus korrekt?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja Vertausche x und y $x = x - y$ Nein $\emptyset$	
Gib den Wert von y als ggT aus	

Korrektheitsüberlegung

In jedem Schleifendurchlauf wird x und ggf. auch y verändert. Durch die Subtraktion der kleineren von der größeren Zahl werden die Werte sukzessive kleiner.

Die Schleife bricht ab, sobald x kleiner oder gleich 0 wird.

→ Der Algorithmus **terminiert**.

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus korrekt?

Korrektheitsüberlegung

Seien x_t bzw. y_t die Werte vor dem t . Schleifendurchlauf und x_{t+1} bzw. y_{t+1} die Werte nach dem t . Schleifendurchlauf.

Schleifeninvariante

$ggT(x_t, y_t) = ggT(x_{t+1}, y_{t+1})$, d.h. der ggT wird durch die Schleife nicht verändert.

Beweis

Sei $g = ggT(x_t, y_t)$, dann gilt

$$x_t = g \cdot a$$

$$y_t = g \cdot b$$

O.B.d.A.: Sei $x_t > y_t$, dann

$$x_{t+1} = x_t - y_t$$

$$= g \cdot a - g \cdot b$$

$$= g \cdot (a - b)$$

$$y_{t+1} = y_t = g \cdot b$$

→ g ist weiterhin ein Teiler von x_{t+1} und y_{t+1}

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus korrekt?

Korrektheitsüberlegung

Seien x_t bzw. y_t die Werte vor dem t . Schleifendurchlauf und x_{t+1} bzw. y_{t+1} die Werte nach dem t . Schleifendurchlauf.

Schleifeninvariante

$ggT(x_t, y_t) = ggT(x_{t+1}, y_{t+1})$, d.h. der ggT wird durch die Schleife nicht verändert.

Beweis (Fortsetzung)

Es kann keinen neuen Teiler $\hat{g}$ geben, der x_{t+1} und y_{t+1} teilt und größer ist als g (und somit ein neuer ggT), denn sonst:

$$\begin{aligned}y_{t+1} &= \hat{g} \cdot \hat{b} = y_t \\x_{t+1} &= \hat{g} \cdot \hat{a} = x_t - y_t = x_t - \hat{g} \cdot \hat{b} \\x_t &= \hat{g} \cdot \hat{a} + \hat{g} \cdot \hat{b} = \\&= \hat{g} \cdot (\hat{a} + \hat{b})\end{aligned}$$

D.h. $\hat{g}$ wäre bereits ein Teiler von x_t und y_t gewesen und damit g nicht der ggT $\rightarrow$ Widerspruch!

q.e.d

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus korrekt?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$	
Ja	Nein
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	

Korrektheitsüberlegung

- Der Algorithmus terminiert.
- Der ggT wird durch die Schleife nicht verändert.
- Beim letzten Schleifendurchlauf galt

$$x = y$$

→ Der ggT zweier gleicher Zahlen ist der Wert dieser Zahlen

q.e.d.

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus effizient?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja Nein	
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	

Um die Effizienz von Algorithmen vergleichen zu können, benötigen wir ein **abstraktes Maschinenmodell**, anhand dem der nötige Speicher und die Anzahl der ausgeführten Befehle bestimmt werden können.

Die Nutzung eines realen Rechners und die Messung einer Implementierung sind nicht zielführend, weil sich die Aussagen bei Weiterentwicklungen der Architektur oder der Programmiersprache ändern könnten.

Abstraktes Maschinenmodell

RAM (Random Access Machine)

Wir verwenden für die Effizienzbetrachtung eine RAM (Random Access Machine) als abstraktes Maschinenmodell.

- Sie besitzt eine unendliche Anzahl von Speicherzellen
- Jede Speicherzelle kann einen Integer unbegrenzter Größe speichern
- Der Befehlssatz der RAM umfasst lesende und schreibende Zugriffe auf die Speicherzellen, Vergleiche von Speicherinhalten und Sprünge
- Jeder dieser Befehle benötigt eine gewisse Laufzeit
- Die Laufzeitkomplexität eines Algorithmus ergibt sich aus der Anzahl der ausgeführten Befehle
- Die Speicherkomplexität ergibt sich aus der Anzahl der benutzten Speicherzellen

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus effizient?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja Vertausche x und y Nein $\emptyset$	
$x = x - y$	
Gib den Wert von y als ggT aus	

Effizienzüberlegung zum Speicherbedarf (Speicherkomplexität)

Es werden nur Speicherzellen für die beiden Eingabewerte benötigt, die dann im weiteren Verlauf wiederverwendet werden.

→ Mit weniger Speicher geht es wohl nicht, daher effizient.

Eigenschaften von Algorithmen

Ist der Euklid-Algorithmus effizient?

Euklid-Algorithmus

Lies zwei ganze Zahlen in x und y ein	
Solange $x > 0$	
$x < y ?$ Ja $x < y ?$ Nein	
Vertausche x und y	$\emptyset$
$x = x - y$	
Gib den Wert von y als ggT aus	

Effizienzüberlegung zur Laufzeit (Laufzeitkomplexität)

Im schlimmsten Fall ist eine der beiden Zahlen 1. Dann wird bei jedem Schleifendurchlauf die größere Zahl ermittelt (1 Vergleich) um 1 dekrementiert (1 Subtraktion).

Es ergibt sich also im Worst Case:

$\max(x, y)$ Vergleiche und $\max(x, y)$ Subtraktionen

Ob dies effizient ist, kann man erst beurteilen, wenn man alternative Algorithmen in einer ähnlichen Form untersucht hat.

Was passiert im Praktikum?

Python

Hinweise zur Nutzung

- Verfügbar für Linux, Mac OS und Windows
- Wir verwenden die Version 3.12 oder neuer
- Viele Algorithmen und Datenstrukturen der Vorlesung sind in der Python-Bibliothek bereits enthalten. Diese verwenden wir aber nicht für unser Praktikum! Auch einfache Listen sind verboten! Stattdessen sind alle unsere Werte Instanzen der Klasse `AlgoDatValue` und Arrays nutzen die Klasse `AlgoDatArray`.

Python

Hinweise zur Nutzung

```
def euklid_ggt(x, y):  
    while x > 0:  
        if x < y:  
            (x, y) = (y, x)  
        x -= y  
    return y
```

```
if __name__ == "__main__":  
    print(euklid_ggt(AlgoDatValue(12), AlgoDatValue(8)))  
  
    AlgoDatValue.summary()
```

Warum?

Wir machen uns zwar das Leben damit zunächst etwas schwerer, können aber die Effizienz besser beurteilen, weil keine „Magie“ in den Bibliotheken stattfindet.

Python

Hinweise zur Nutzung

```
def euklid_ggt(x, y):  
    while x > 0:  
        if x < y:  
            (x, y) = (y, x)  
        x -= y  
    return y  
  
if __name__ == "__main__":  
    print(euklid_ggt(AlgoDatValue(12), AlgoDatValue(8)))  
  
    AlgoDatValue.summary()
```

Ausgabe:

```
4  
Memory: 2  
Read: 18  
Write: 5  
Compare: 7  
Add: 0  
Sub: 3
```


Vielen Dank für die Aufmerksamkeit!